

INFORME DE PROYECTO DE SOFTWARE

SISTEMA WEB VETIDAT

Digitalización y optimización de los procesos de atención clínica veterinaria

CURSO: Interfaces de Usuario III
CICLO: IV Ciclo
PROYECTO: Aplicación Web Standalone en Angular y TypeScript
GITHUB: <https://github.com/ronyemerzon/vetidat>

INTEGRANTES DEL GRUPO:

1. Rony Emerzon Pascual Cardenas
2. Alison Laulate Villa
3. Cristhian moreno Silva

1. INTRODUCCIÓN Y CONTEXTO

La digitalización y el diseño de interfaces de usuario eficientes se han convertido en un pilar clave para el crecimiento de cualquier negocio en la actualidad. En el sector de la salud animal, el flujo ágil de la información médica y administrativa influye de forma directa en el bienestar de los pacientes y en el nivel de satisfacción de los clientes.

1.1. Situación Problemática

Una clínica veterinaria en pleno crecimiento ha reportado pérdidas económicas y reclamos constantes debido a la ineficiencia de sus procesos analógicos. Entre los problemas críticos identificados se encuentran:

- Proceso de agenda lento al depender únicamente de llamadas telefónicas manuales.
- Falta de visibilidad sobre los horarios de atención de médicos disponibles.
- Carencia de un historial de atenciones y consultas médicas centralizado para las mascotas.
- Pérdida de citas e imposibilidad de planificar preventivamente tratamientos.

1.2. Objetivos del Sistema

El sistema web "VetIDAT" ha sido concebido para resolver estos puntos de dolor a través de las siguientes soluciones informáticas:

- Módulo de Mascotas: Formulario de registro unificado para nuevas mascotas y dueños con validaciones avanzadas.
- Módulo de Citas: Calendario de agenda interactiva donde se ordenan y gestionan los estados de las atenciones.
- Módulo de Historial Clínico: Consulta de la línea de tiempo de atenciones y registro de nuevos eventos.
- Directivas y Pipes: Elementos de UX para alertar y formatear la información para los médicos veterinarios.

2. ARQUITECTURA MODULAR DE ANGULAR

El proyecto ha sido desarrollado bajo la arquitectura modular y moderna de Angular, utilizando el estándar de Standalone Components. La separación lógica de responsabilidades por dominios asegura la escalabilidad y facilidad de mantenimiento del software.

2.1. Estructura de Directorios del Código Fuente

src/app/	
%%% core/	# Modelos POO y servicios centralizados
% %%% models/	# Clases tipadas (Persona, Dueno, Mascota, etc.)
% %%% services/	# VetDataService (Persistencia LocalStorage)
%%% modules/	# Módulos y vistas independientes por dominio
% %%% mascotas/	# Registro y listado de pacientes
% %%% citas/	# Calendario interactivo de atenciones
% %%% historial/	# Consulta e historia clínica por mascota
%%% shared/	# Pipes, Directivas y Componentes reutilizables

2.2. Capa de Datos y Persistencia Local (LocalStorage)

Para garantizar que la aplicación web funcione de forma interactiva e independiente de un servidor externo durante la evaluación, se ha diseñado el servicio "VetDataService". Este servicio centraliza el estado del sistema, guardando de forma persistente a los dueños, las mascotas y las citas médicas en el LocalStorage del navegador del usuario.

3. IMPLEMENTACIÓN DE POO Y TYPESCRIPT

Uno de los requerimientos más importantes de la rúbrica (Criterio 1) es la aplicación de la Programación Orientada a Objetos en TypeScript. Para alcanzar la excelencia, el código implementa los cuatro pilares fundamentales:

3.1. Abstracción

Se definen clases abstractas que actúan como plantillas estructurales y no se pueden instanciar de manera directa. Ejemplos de esto son la clase "Persona" y la clase "RegistroClinico" (base para el historial médico).

3.2. Herencia

La reutilización de código se demuestra a través de las extensiones de clases:

- La clase "Dueno" hereda de "Persona" añadiendo atributos residenciales y la relación de mascotas.
- Las clases "RegistroVacuna", "RegistroTratamiento" y "RegistroCirugia" heredan de la clase abstracta "RegistroClinico" adquiriendo sus atributos base de fecha, veterinario y diagnóstico.

3.3. Polimorfismo

Se implementa mediante el método abstracto "generarResumenDetallado()" en la clase base "RegistroClinico". Cada subclase implementa una lógica particular:

- Vacunas: Imprime detalles de lote y fecha de próxima dosis.
- Tratamientos: Formatea las dosis médicas y el tiempo de tratamiento.
- Cirugías: Formatea los tipos de anestesia y precauciones postoperatorias.

En la UI del historial, Angular itera sobre un arreglo general de "RegistroClinico[]" e invoca el mismo método, ejecutando dinámicamente la función correspondiente de cada clase hija.

4. ELEMENTOS DE ANGULAR Y UX (CRITERIO 4)

4.1. Pipes Personalizados

- Pipe "estadoCita": Traduce los estados técnicos de las citas a etiquetas amigables en español (ej. "pendiente" a "Pendiente").
- Pipe "fechaAmigable": Toma cadenas estándar YYYY-MM-DD y las convierte en fechas naturales completas en español (ej. "2026-07-08" a "Miércoles, 8 de Julio de 2026"), logrando una visualización refinada en el calendario.

4.2. Directiva Personalizada "appResaltarProxima"

Esta directiva evalúa dinámicamente el día en que ocurrirá la cita con respecto a la fecha actual:

- Si la cita es para hoy, pinta de color rojo la alerta visual izquierda y genera una sombra de pulso.
- Si la cita es para mañana, utiliza el color naranja institucional para alertar visualmente al usuario.

Esto optimiza enormemente la experiencia del usuario y cumple con los requerimientos técnicos avanzados de la rúbrica.

4.3. Integración de Bootstrap 5

Se instaló Bootstrap mediante npm y se configuró a nivel global en el compilador de Sass. La aplicación utiliza:

- El sistema de grilla responsiva (Grid System) para reordenar la información en pantallas móviles.
- Componentes pre-diseñados como badges de estado, alertas de éxito y cajas de formulario reactivos con clases específicas para mejorar la estética visual y la retroalimentación inmediata al personal.

5. INSTALACIÓN Y PLAN DE VERIFICACIÓN

5.1. Pasos para la Instalación Local

1. Ingresar a la carpeta del proyecto

```
cd ProyectoInterfacesIII
```

2. Instalar dependencias del package.json

```
npm install
```

3. Levantar servidor local de desarrollo

```
npm run start
```

5.2. Flujo de Pruebas Recomendado

Para validar que el proyecto cumple con la rúbrica al calificar, sugerimos realizar las siguientes acciones en orden:

1. Navegar entre las pestañas del Navbar para verificar la velocidad de la SPA (Rutas sin recargas).
2. Registrar un dueño nuevo y su mascota en /mascotas. Comprobar que los validadores alerten de formatos de teléfonos incorrectos.
3. Verificar la adición inmediata de la mascota en la sección derecha (Persistencia LocalStorage).
4. Ir a /citas y comprobar el resaltado rojo (para citas programadas hoy) y naranja (para mañana) a través de la directiva personalizada.
5. Verificar el funcionamiento del pipe "estadoCita" traduciendo estados en las badges de la agenda.
6. Ingresar a /historial y seleccionar a la mascota creada. Podrá verificar los historiales cargados de forma dinámica y la diferenciación de atenciones (Polimorfismo POO).

